

DESIGN DOCUMENT

AUDIO FORMAT

Document History:

S.No	Version No.	Author	Section(s) Affected	Change(s) made	Approved Date	Approved by
1	1.0	Nithishkumar.T	All	Initial		
2	1.1	Nithishkumar.T	Low Level Design	Design image		

Table of Contents

1. INTRODUCTION.....	3
2. PROBLEM STATEMENT	3
3. OBJECTIVE	3
4. DESIGN PLAN	4
4.1 SYSTEM OVERVIEW	4
4.2 DESIGN.....	4
4.2.1 HIGH LEVEL DESIGN	4
4.2.2 LOW LEVEL DESIGN	5

1. INTRODUCTION

This project is about the conversion of the given audio sample input into various formats of data. Audio data often needs to be transferred between different representations, such as block-based and interleaved formats, depending on the application or processing requirements. Block-based formats store each channel's samples in separate, contiguous blocks, while interleaved formats arrange the samples of all channels sequentially within the same buffer. Efficient conversion between these formats is crucial for optimizing memory usage and computational performance in real-time audio applications. The goal of this project is to design a API that efficiently handles these format conversions, ensuring minimal latency and resource consumption in diverse environments, from embedded systems to high-performance audio workstations.

2. PROBLEM STATEMENT

The problem statement is about developing a single API that supports to move the audio samples from one buffer to another. For that D19API should support multiple formats and the below mentioned format needs to support

- Block to Block
- Block to Interleaved
- Interleaved to Interleaved
- Interleaved to Block

It should have different input arguments like, block size, interleaved stride and the API should be an optimized one in terms of MIPS and Memory

3. OBJECTIVE

The main objective of this is to implement the audio format conversion from the sample audio input with different forms of conversion. The conversion involves Block to Block, Block to Interleaved, Interleaved to Interleaved and Interleaved to Block. For each conversion the previous output data is got to the input of the conversion, and the format needs to be changed.

- **Block to Block**

This operation transfers audio samples from one block-based buffer to another, preserving the block structure. Each block contains the samples of a single channel, and the operation ensures that the data is efficiently moved while maintaining the original layout.

- **Block to Interleaved**

In this transformation, audio data stored in a block-based format (where each channel has its own buffer) is converted to an interleaved format (where the samples of all channels are interspersed in a single buffer). This operation requires careful mapping of samples from each channel into a continuous buffer with the correct stride for each channel.

- **Interleaved to Interleaved**

This conversion allows the transfer of audio data between two interleaved buffers. This can be particularly useful in scenarios where the stride or the number of channels may differ between the two buffers, and the data must be adjusted accordingly. The operation involves ensuring proper reorganization and alignment of the data to match the target buffer's interleaving pattern.

- **Interleaved to Block**

This operation converts interleaved audio data into a block-based format, where each channel's data is separated into distinct blocks. The samples in the interleaved buffer must be extracted and reallocated to individual block buffers corresponding to each channel.

4. DESIGN PLAN

4.1 SYSTEM OVERVIEW

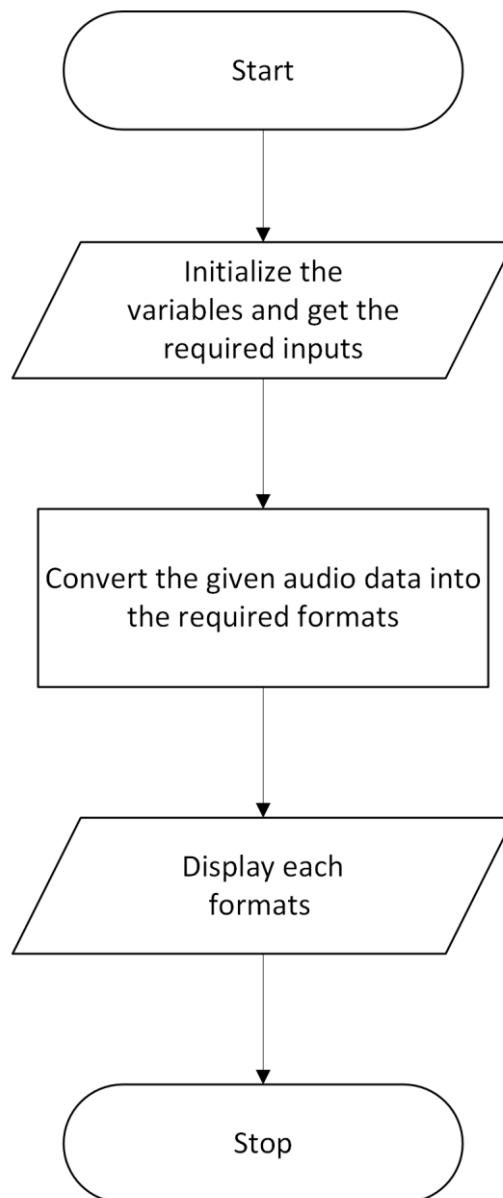
The System changes the input of the audio file from the given sample audio so that we can convert the required formatted files. This will be done by making the identification of the block size and the interleaved stride size which are made by the user that can be available from the source data. The code needs to be optimized in that system and the memory management also viewed. Based on this the optimization code is developed. The working of the conversion is described in the following design.

4.2 DESIGN

4.2.1 HIGH LEVEL DESIGN

In the High-Level design, the block explains the overall way of approaching the given problem statement. The design begins with initializing the necessary variables and gathering required inputs, such as the audio file to be processed and any preferences for the output format. After the setup, the next step is to convert the input audio data into the desired formats. This involves reading the input audio file, processing the data and then encoding the audio into the specified formats. Once the conversion is complete, the

resulting files are displayed or logged, showing details of the format file. Finally, the process ends with the program stopping, potentially saving the converted files and cleaning up any resources. The High-Level Design is attached as follows.



4.2.2 LOW LEVEL DESIGN

In the Low-Level design, each block explains the working way of approaching the given problem statement. In that each data is transferred to the input of another data and converted based on the required format.

The low-level design outlines a process for transforming data between different formats, specifically focusing on converting between block data format and interleaved data format across four distinct modes.

The process begins with obtaining the input file, which contains data that needs to be transformed. Before performing any transformation, two crucial parameters are gathered, the block size and the interleaved stride. These parameters will dictate how the data is structured and organized during the conversion process. The block size typically refers to the fixed size of data chunks in the block format, while the interleaved stride likely represents how the data is distributed across multiple channels or arranged in a pattern for the interleaved format.

- **Mode 1: Block to Block**

In Mode 1, the input file is transformed from its original format (assumed to be unformatted or raw) into a specific block data format. Block data format organizes data into fixed-size blocks, and this mode is about converting the data into such blocks. Once the data is converted, it is copied into a separate buffer designed to hold the block data format. This step ensures that the original data is preserved while the converted data is available for further processing. The converted block data is then displayed, giving an overview of how the data looks after the transformation.

- **Mode 2: Block to Interleaved**

After completing Mode 1, Mode 2 takes the buffer generated in Mode 1 (containing data in block format) and transforms it into an interleaved data format. The interleaved format typically refers to a method where data elements are distributed across multiple streams, channels, or memory locations in a non-contiguous manner. This step might involve reordering or redistributing the data across different elements or dimensions based on the interleaving pattern. The transformed data is then stored in a new buffer designed for interleaved data. Once again, the resulting buffer is displayed, allowing for verification and inspection of the interleaved format.

- **Mode 3: Interleaved to Interleaved**

In Mode 3, the transformation involves working with data that is already in interleaved format. Here, the interleaved data might undergo further manipulation, which could involve changing the interleaving pattern, reordering the data, or applying some other form of transformation that redefines how the data is interleaved. This step produces another buffer that stores the modified interleaved data format. Like the previous modes, the transformed data is displayed to

provide a visual representation of how the interleaved data has been altered.

- **Mode 4: Interleaved to Block**

Finally, in Mode 4, the interleaved data (produced in Mode 3) is transformed back into the block data format. This is a reverse operation of Mode 2, where the interleaved format is de-interleaved and restructured back into a series of fixed-size blocks. The transformed data is stored in yet another buffer that holds the block format. The final transformed block data is displayed, providing an opportunity to review the output of the entire transformation pipeline.

At the end of the process, the system has gone through a full cycle of data transformations: from raw input to block, then to interleaved, back to interleaved, and finally back to block format. This allows for flexibility in handling different data formats and ensures that the data can be represented in multiple ways based on the requirements of the system or application.

The process is a series of transformations that take input data and progressively convert it between two formats: block format and interleaved format. Each transformation step is designed to preserve the integrity of the data while converting it into a different organizational structure, with clear intermediate buffers storing the data in each stage. Each step ends with a display of the converted data, offering the user the chance to verify the correctness of the transformations. This process is useful in scenarios like data processing, multimedia applications, or any system that requires data to be manipulated across different representations for optimization or compatibility reasons. The Low-Level Design is attached as follows.

